Terminal ×

```
codio@dolphinsupreme-portallorenzo:~/workspace$
```

Guide ×

Collapse

Building a Wheel

Wheel Types

There are three types of wheels:

- Universal wheel - this wheel can run on either Python 2 or Python 3
- Pure-Python wheel - this wheel only runs on Python 2 or Python 3, not both
- Platform wheel - this wheel is designed for a specific version of Python and platform (process and/or operating system)

A quick note on Python 2, this version of Python is no longer being officially supported, so any discussion of wheels will not focus on Python 2. We are also not going to get into specifics of platform-specific wheels. You can look at the Python packaging user guide for more information.

Instead, we are going to focus on building a pure-Python wheel for Python 3.

Setup

In a previous assignment, we installed the `shapes` package from GitHub. We are going to create a wheel for this package. This package already resides on the system, there is no need to get it again from GitHub.

The package resides in the `shapes-package` directory. Change into this directory first. Then create a virtual environment called `shapes-env`. Finally, activate the virtual environment.

```
cd shapes-package
python3 -m venv shapes-env
source shapes-env/bin/activate
```

Update `pip`, `wheel`, and `setuptools`. These packages should be updated just to make sure that the process goes smoothly.

```
python -m pip install -U pip wheel setuptools
```

You need a `setup.py` file. This file is used to set features and options for the desired wheel. Click on the link below to open the setup file for the `shapes` package.

Open setup.py

Start by importing `setup` from `setuptools`. Then we are going to pass named arguments to the `setup` function. At the very least, your `setup.py` file needs values for:

- `name` - the name of your package
- `version` - the version of your package
- `packages` - the list of where you can find the Python files for the package

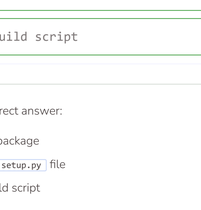
Here is a sample `setup.py` file that lists all of the possible options that can be set for the wheel. Our wheel is going to add a description and an author. There is a directory called `shapes` that contains the actual Python package. Put this name in a list for the `packages` option.

```
from setuptools import setup

setup(
   name='shapes',
   version='0.1',
   description='Package for calculating area and perimeter for circles, squares, ar
   author='Your name here',
   packages=['shapes'],
)
```

Building the Wheel

Now that the `setup.py` file is ready, we need to change into the `shapes` directory. The image below shows the hierarchy for `shapes-package`.



The following command takes the source distribution (found in the next `shapes` directory) and creates a build distribution in the form of a wheel.

```
python3 setup.py sdist bdist_wheel
```

The build process created the `dist` directory. Change into this directory and then use the `ls` command, which lists the contents of the directory.

```
cd dist
ls
```

You should see the following output. The file that ends with `.whl` is the wheel. Notice that the file name for the wheel tells us lots of useful information. `shapes-0.1` is the name and version number. `py3` means the wheel works on Python 3. The `none` relates to the application binary interface (ABI). Our package does not interact with the ABI, which is why Python uses `none`. Finally, the `any` means the package can run on any platform. The `.tar.gz` file is the zipped source distribution of the package.

```
shapes-0.1-py3-none-any.whl
shapes-0.1.tar.gz
```

Reading Question

Place the following steps for creating a wheel in order.

Create the package

Create the setup file

Run the build script

Here is the correct answer:

- Create the package
- Create the `setup.py` file
- Run the build script

The first thing to do is write all of the Python code for the package. The next step is to create the `setup.py` file. Finally, run the build script to generate the wheel.

Check It!

Next